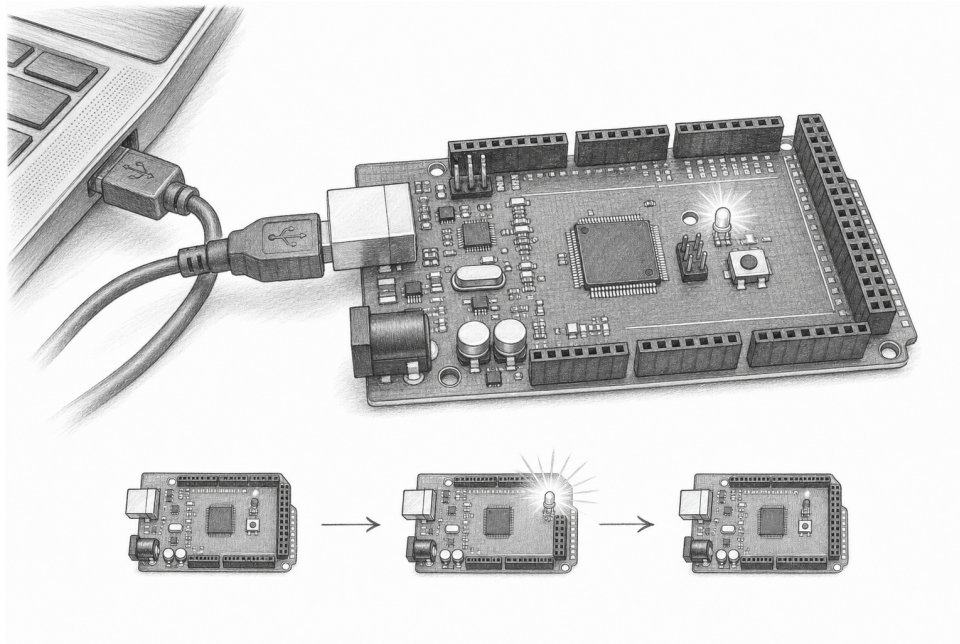


Make one light become evidence

Lesson 001: the Mega 2560 built-in LED

Predict → command → observe → explain



Teaching job of the plate. Use this graphite view only to orient the board, USB cable, and changing light. It is an observational illustration, not a pin map or circuit measurement. Its prominent lamp is symbolic: the Mega 2560 does not require the learner to find, touch, or wire a literal through-hole LED shown in this plate. The exact logic used in this lesson appears in the vector diagrams below. The Mega's built-in indicator is conventionally addressed by `LED_BUILTIN`; do not infer a pin from the drawing.

Question	Can a C++ object make the board's own LED produce a repeatable, testable timing pattern?
Time	35–50 minutes, plus one controlled extension
You need	Arduino Mega 2560, known-good USB data cable, Arch Linux computer, <code>arduino-cli</code> , this ADK checkout
You do not need	Breadboard, jumper wires, resistor, external LED, battery, or RF equipment
Proof of success	At least three complete cycles observed, a timing record, and a claim supported by evidence and reasoning

Safety boundary and stop conditions

This lesson uses only the board's factory-installed LED and USB power. Work on a dry, nonconductive surface. Remove jewelry that could bridge conductors. Keep loose metal away from headers. Never connect USB and an unknown external power source at the same time.

All-power-off setup check: disconnect USB, barrel-jack power, anything on VIN, batteries, shields, and every external lead before inspection. Do not assume removing USB de-energizes a previously assembled board. Confirm that no power source remains, then inspect the bare board and cable. Proceed only if the board is dry and has no scorched area, loose metal, bent pins touching one another, or damaged USB connector.

Stop immediately if a component becomes hot, smoke or odor appears, the power LED flickers unexpectedly, the USB connector is loose, or the host reports repeated over-current disconnects. Remove power at its upstream source—USB at the computer end and any other source, if one is unexpectedly present and can be disconnected safely. Do not touch a hot or damaged board and do not “try once more.” Mark it DO NOT USE and escalate to a qualified maintainer.

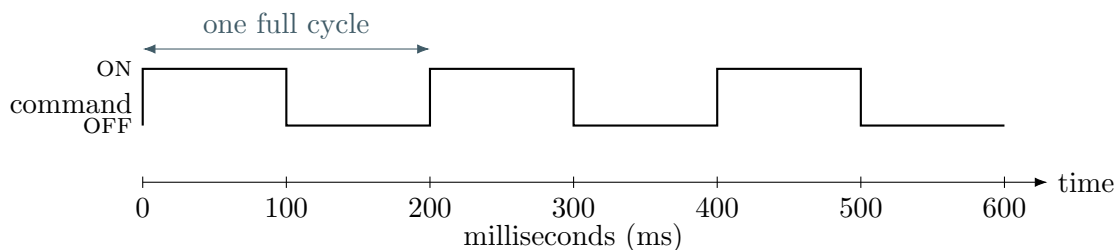
Check — record the all-power-off inspection before applying power

Board ID or label: _____ Date/time: _____

☐ every power source removed ☐ bare board dry ☐ no visible damage ☐ headers clear ☐ cable sound

1. Predict before running

The supplied program commands the LED on for 100 ms and off for 100 ms. Complete the prediction without uploading.



Prediction A. One full cycle lasts _____ ms. In seconds this is _____ s.

Prediction B. Expected cycles per second:

$$f = \frac{1}{T} = \frac{1}{\text{_____ s}} = \text{_____ cycles/s.}$$

Prediction C. During a 10 s observation, I expect approximately _____ complete cycles. I expect the LED to appear ☐ clearly blinking ☐ steadily lit ☐ steadily dark.

Commit before revealing the worked example: Why might your eyes give weaker timing evidence than the program text?

Worked example, with units

The two delays make

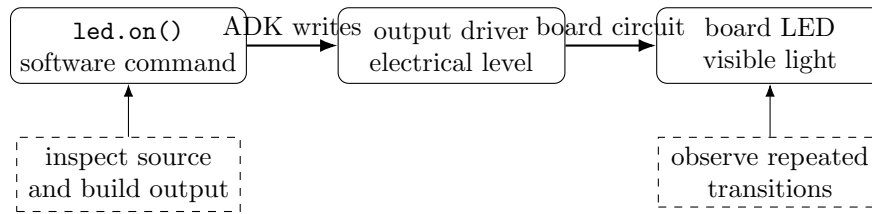
$$T = 100 \text{ ms} + 100 \text{ ms} = 200 \text{ ms} \left(\frac{1 \text{ s}}{1000 \text{ ms}} \right) = 0.200 \text{ s.}$$

Therefore

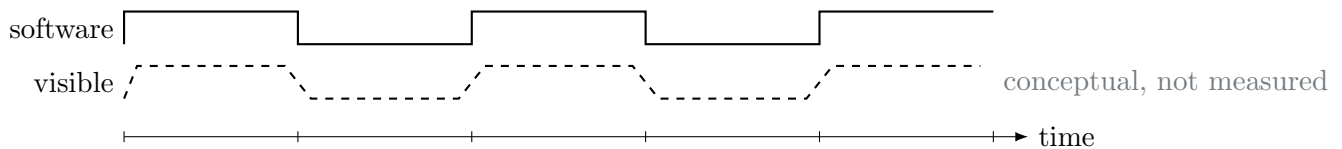
$$f = \frac{1}{0.200 \text{ s}} = 5.00 \text{ s}^{-1} = 5.00 \text{ Hz.}$$

Ten seconds should contain about $10 \text{ s} \times 5.00 \text{ s}^{-1} = 50$ cycles. That is the *commanded* pattern. The time spent executing calls and loop instructions, the board clock’s tolerance, and human reaction time make a hand-timed observation less exact.

2. Three representations of the same experiment



Important limitation. Seeing light supports the claim that the visible indicator changed. It does not independently measure pin voltage, exact duration, or current. This no-external-wiring lesson intentionally does not probe those quantities.



The dashed visible trace illustrates finite response and perception; it is not sample data. What observation would weaken the explanation that the program causes the pattern? For example: the LED continues unchanged after the program is replaced by a known-dark sketch.

3. The lifecycle in the supplied program

```

#include <adk.h>

adk::led::Mono led(LED_BUILTIN);

void setup()
{
    adk::initialize();
}

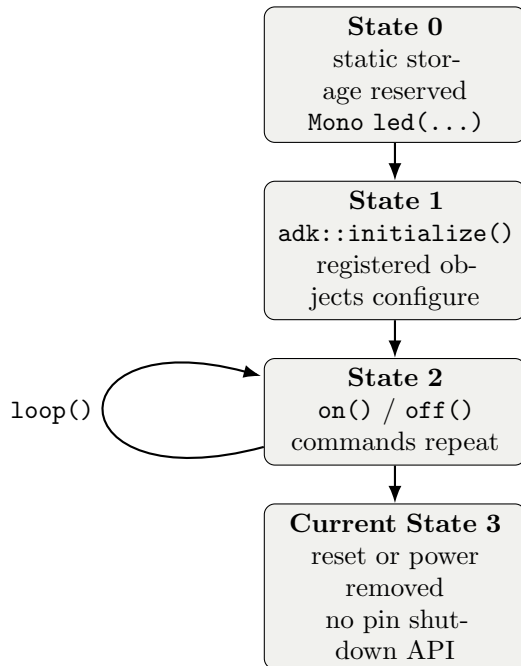
void loop()
{
    constexpr auto intervalMs = 100;

    led.on ();
    delay  (intervalMs);

    led.off ();
    delay  (intervalMs);
}
  
```

Opening parentheses are aligned within the logical group. Braces occupy their own lines, and the ADK component is a **struct**.

Read the object lifetime, not just the blink



Why explicit initialization?

Construction identifies the resource; initialization performs the fallible or platform-dependent hardware setup at a deliberate time. **Target design, not current proof:** ADK will not use exceptions internally. An owned resource will clean up safely during ordinary destruction or external exception unwinding. Component lifecycle APIs will be named `initialize()` and `shutdown()`, avoiding collision with STL iterator `begin()` and `end()`.

Current implementation versus target. Today, the global `adk::initialize()` helper initializes registered objects. `Object`'s destructor unregisters an object, but the current pin and LED types do not yet expose component-level `shutdown()` or demonstrate pin-state cleanup. The target RAII design will add idempotent, nonthrowing cleanup: generic outputs become high-impedance by default and semantic LEDs become inactive. This lesson demonstrates current initialization and commands; it does *not* claim that the target cleanup behavior exists yet.

Check — explain each line before upload

Line or construct	Your explanation
<code>LED_BUILTIN</code>	
<code>adk::initialize()</code>	
<code>led.on()</code>	
<code>delay(intervalMs)</code>	

4. Build without touching the board

From the repository root, run:

```

arduino-cli version
arduino-cli core list
make arduino-001
  
```

The repository defaults `BOARD_FQBN` to `arduino:avr:mega`. If you override it, record the override. A successful compile is evidence that the translation and link stages accepted the program for the selected board; it is not evidence that upload or hardware behavior succeeded.

Check — State 0 — compile acceptance**Record, do not paraphrase:**

Board/FQBN

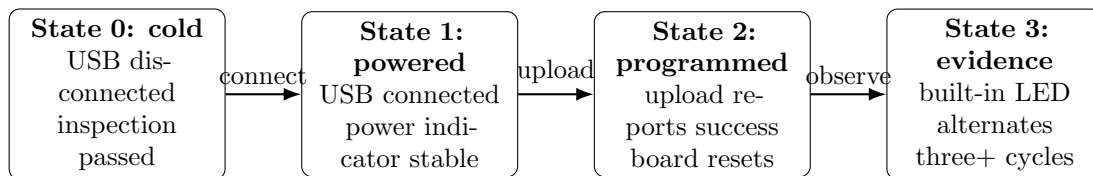
Tool version

Final success line

Flash bytes / capacity

RAM bytes / capacity

Reject the build if the board target is not the Mega 2560, compilation has any error, or size exceeds the reported capacity. Fix that evidence before uploading.

5. Perform and verify: numbered physical states

1. Place the inspected board on a dry, nonconductive surface. Keep all headers empty. Verify State 0 against the diagram.
2. Connect the USB cable. Verify the ordinary power indicator is stable. If not, disconnect and stop.
3. Identify the serial port with the repository's documented listing command. Record the exact port: _____.
4. Upload with `make upload LESSON=001 PORT=/dev/ttyACM0`, replacing the example port with the exact port recorded in step 3. Read the final status; do not treat scrolling text as success.
5. After automatic reset, observe from a comfortable distance. Count *complete* bright–dark cycles for 10 s. Repeat three times.

Check — intermediate question before interpreting

Does the LED visibly reach both a brighter and a darker state? ☐ yes ☐ no. Does the upload tool explicitly report success? ☐ yes ☐ no.

If either answer is no, do not fill the acceptance claim yet; use the troubleshooting table.

Table 1: Direct observations. Enter your data; the expected pattern is not sample data.

Trial	Observation time (s)	Complete cycles	Notes: brightness, irregularity, reset
1	10		
2	10		
3	10		

Mean observed count:

$$\bar{n} = \frac{n_1 + n_2 + n_3}{3} = \frac{\text{---}}{3} + \frac{\text{---}}{3} + \frac{\text{---}}{3} = \text{---} \text{ cycles per 10 s.}$$

Range = $\max(n) - \min(n)$ = --- cycles.

6. Diagnose by symptom, with a safe next check

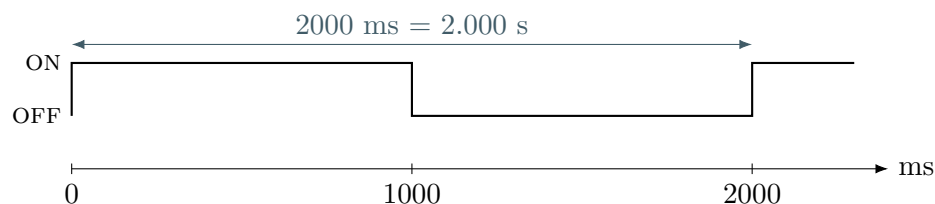
Symptom	Plausible cause	Safe next check
No power indicator	Charge-only/damaged cable, bad USB port, damaged board	Disconnect. Inspect and try one known-good data cable and host port. Stop if heat, odor, or repeated disconnect occurs.
Port absent	Permissions, cable, or device enumeration	Keep headers empty. Run the port-listing command before and after connection; compare the lists. Do not guess a device path.
Compile fails	Wrong target, missing core, source/API mismatch	Stay disconnected. Record the first error, selected FQBN, and tool versions. Fix the first error before upload.
Upload fails	Wrong port/FQBN, port in use, cable problem	Do not repeatedly reset. Confirm Mega 2560 target, close serial monitors, re-list ports, retry once with recorded settings.
Upload succeeds; LED dark	Wrong observable LED, command not reached, board variant, hardware fault	Press reset once and watch from reset onward. Re-read <code>LED_BUILTIN</code> ; upload a known-good built-in LED example only as a controlled comparison.
LED steady	Blink too fast to resolve, stale program, reset loop	Confirm the reported upload artifact and port. Change only the interval to 1000 ms for the extension; observe whether clear transitions appear.
Irregular pattern	Reset, unstable USB, host connection, observation error	Check whether power indicator or serial connection also resets. Repeat a timed trial without touching cable or board.

Escalate rather than improvise when two known-good cables/ports still cannot enumerate the board, a known-good example cannot control the built-in LED, physical damage is visible, or any electrical stop condition occurs.

7. Controlled extension: change one variable

Change only the interval from 100 ms to 1000 ms. Keep the same board, port, cable, component, sequence, observer, and 10 s observation window.

```
constexpr auto intervalMs = 1000;
```



Before uploading, predict the new frequency:

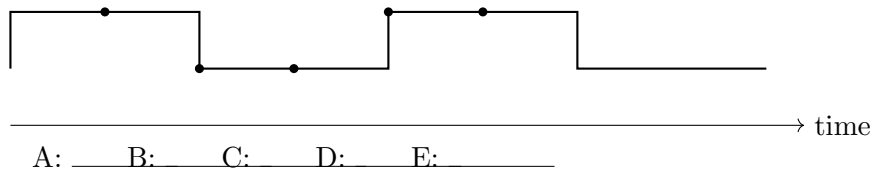
$$f = \frac{1}{(1000 + 1000) \text{ ms}} = \text{_____ Hz.}$$

Condition	Predicted cycles / 10 s	Observed cycles / 10 s	What changed perceptually?
100 ms interval			
1000 ms interval			

Controlled-comparison inference: If the longer interval creates fewer, more easily distinguished transitions, the changed interval is the leading explanation because the other named variables were held constant. It is not proof that every timing source is exact.

8. Assessment: can you transfer the method?

A. Label the states. Write COMMANDED ON, COMMANDED OFF, or TRANSITION beneath each marked point.



B. Calculate. For 250 ms on and 750 ms off, calculate period, frequency, and duty fraction. Keep units visible.

C. Separate evidence from inference. Mark each statement O (observation), I (inference), or U (unsupported by this lesson).

- _____ The visible LED alternated between brighter and darker states.
- _____ Exactly 5.000000 cycles occurred every second.
- _____ The uploaded program is a plausible cause of the repeated pattern.
- _____ The output pin delivered a measured 5.00 V.

D. Lifecycle. Why is `initialize()/shutdown()` clearer for hardware ownership than `begin()/end()` in a C++ library?

E. Falsification. Name one result that would weaken your explanation that the ADK commands caused the visible pattern.

9. Final acceptance proof: Claim–Evidence–Reasoning

Acceptance criteria

- ☐ cold inspection passed ☐ Mega 2560 target recorded ☐ compile and upload explicitly succeeded
☐ both visible states observed ☐ three timed trials recorded ☐ controlled extension changed only the interval

Claim. State narrowly what this build demonstrated. Do not claim voltage or exact timing unless independently measured.

Evidence. Cite your upload result, repeated counts (with units), and controlled comparison.

Reasoning. Explain why those observations support the claim and why the one-variable comparison matters.

Uncertainty and alternatives. State at least one limitation (human counting, oscillator tolerance, hidden board circuitry, reset behavior) and one plausible competing explanation.

Lesson complete only when the evidence survives review. A reviewer should be able to reconstruct what target was built, what was observed, how often it was repeated, what variable changed, and what would cause rejection. A bright LED by itself is an event; a documented, repeatable comparison is evidence.

Instructor / reviewer key

Expected values: 100 ms case $T = 200 \text{ ms} = 0.200 \text{ s}$, $f = 5.00 \text{ Hz}$, approximately 50 cycles in 10 s; 1000 ms case $T = 2.000 \text{ s}$, $f = 0.500 \text{ Hz}$, approximately 5 cycles in 10 s. Assessment B: $T = 1.000 \text{ s}$, $f = 1.00 \text{ Hz}$, duty fraction $250/1000 = 0.250 = 25.0\%$. Assessment C: O, U, I, U. Accept reasonable variation in hand counts only when the learner names the uncertainty. Reject a CER that substitutes expected values for recorded observations, claims a measured voltage, or omits the controlled comparison.

Asset provenance note. The graphite orientation plate is the project-provided asset `001-built-in-led-pencil.png`, reviewed here only for board/cable/light orientation. It is not authoritative for component placement, pin identity, dimensions, or electrical topology. All timing, state, lifecycle, and acceptance logic in this publication is rendered as deterministic TeX/TikZ.